Facultad de Ciencias - UNAM
Lógica Computacional 2026-2
Práctica 3: Resolución Binaria

Favio Ezequiel Miranda Perea Patricio Ordoñez Blanco Eduardo Vargas Pérez

10 de Marzo de 2026
Fecha de entrega: 26 de Marzo de 2026

1. Introducción

A lo largo del curso, se ha revisado el concepto de **satisfacibilidad**, y el cómo a base de interpretaciones se puede saber si una fórmula es satisfacible o no.

Lamentablemente, verificar por medio de interpretaciones si una fórmula es satisfacible no es factible. Esto se debe a que se tiene que dar un estado para las variables y comprobar la satisfacibilidad de una fórmula dada, lo que puede llevar a un análisis de muchos estados. En particular nos llevaría a una complejidad de $\mathcal{O}(2^n)$, donde `n` sería el número de variables dentro de la fórmula, lo cual no es para nada óptimo.

Para ello, se introduce el concepto de **Resolución Binaria**, el cual por medio de una regla de inferencia, permite resolver de manera "más óptima" el problema de satisfacibilidad para una fórmula de la lógica proposicional.

2. Objetivo

El objetivo principal de esta práctica es revisar el concepto de Resolución Binaria de manera práctica. En este proceso, se busca que repasen el concepto de satisfacibilidad, refuercen su dominio con las **formas normales** y que implementen un **algoritmo de saturación**.

3. Desarrollo de la práctica

3.1. Lógica proposicional

Para poder trabajar con la lógica proposicional en `Haskell`, hay que realizar una forma de representar las fórmulas. Para esto, se reutilizará el tipo de dato algebraico que se usó en la práctica 2:

```
 data Prop = Var String | Cons Bool | Not Prop
.           | And Prop Prop | Or Prop Prop
.           | Impl Prop Prop | Syss Prop Prop
.           deriving (Eq)
```

Notar que se representan las variables como cualquier cadena, se da soporte a las constantes Booleanas ‘True‘ y ‘False‘, y se tiene un constructor para cada uno de los operadores vistos en clase.

Adicionalmente, para facilitar la lectura de las fórmulas en la terminal, se debe agregar el siguiente código que le dice a *Haskell* cómo debe imprimir un tipo de dato Prop.

```
 instance Show Prop where
.           show Cons True = "⊤"
.           show Cons False = "⊥"
.           show (Var p) = p
.           show (Not p) = "¬`+ show p
.           show (Or p q) = "(`+ show p ++ "∨ `+ show q ++ ")"
.           show (And p q) = "(`+ show p ++ "∧ `+ show q ++ ")"
.           show (Impl p q) = "(`+ show p ++ "→ `+ show q ++ ")"
.           show (Syss p q) = "(`+ show p ++ "↔ `+ show q ++ ")"
```

3.2. Formas Normales

Como se vio en clase, la resolución binaria no trata de manera directa con una fórmula, trabaja con la forma clausular de esta. En Lógica Proposicional, una cláusula se define como una disyunción de literales, en donde una literal es o una variable o la negación de una.

Forma normal negativa

La primera transformación que se debe aplicar a la fórmula es la forma normal negativa (FNN). Es importante recordar que una fórmula proposicional A está en forma normal negativa si:

- A no contiene equivalencias ni implicaciones.
- Las negaciones solo afectan a variables proposicionales.

Cualquier fórmula de la lógica proposicional puede ser convertida a forma normal negativa en tiempo lineal usando las siguientes equivalencias lógicas:

- Eliminación de la doble negación: $\neg\neg A \equiv A$
- Leyes de Morgan: $\neg(A \wedge B) \equiv \neg A \vee \neg B \;\;|\;\; \neg(A \vee B) \equiv \neg A \wedge \neg B$
- Eliminación de conectivos:

$$A \to B \equiv \neg A \vee B$$
$$A \leftrightarrow B \equiv (A \to B) \wedge (B \to A) \equiv (\neg A \vee B) \wedge (\neg B \vee A)$$

Forma normal conjuntiva

La forma normal conjuntiva (FNC) de una fórmula es equivalente a la forma clausular de esta. Una fórmula proposicional A está en forma normal conjuntiva si es una conjunción de disyunciones de literales; es decir, si es una conjunción de cláusulas.

Cualquier fórmula proposicional puede ser convertida a forma normal conjuntiva usando las siguientes equivalencias lógicas.

- Las mencionadas anteriormente para la forma normal negativa.
- Leyes distributivas: $A \vee (B \wedge C) \equiv (A \vee B) \wedge (A \vee C) \ \mid A \wedge (B \vee C) \equiv (A \wedge B) \vee (A \wedge C)$

Forma normal disyuntiva

Es importante recordar que existe otra forma normal llamada forma normal disyuntiva que consiste en la disyunción de conjunciones de literales. Sin embargo, esta forma normal no es de interés para esta práctica.

Ejercicios:

1. `fnn :: Prop → Prop`
   Implementar la función `fnn` que convierte una fórmula proposicional en su forma normal negativa. **[1.5 puntos]**
2. `fnc :: Prop → Prop`
   Implementar la función `fnc`, que convierte una fórmula proposicional en su forma normal conjuntiva. **[1.5 puntos]**
   Hint: Utilicen la función `fnn`.

3.3. Resolución Binaria

La resolución binaria es un método sintáctico equivalente a interpretaciones cuya base es la regla de inferencia que se muestra a continuación.

$$\frac{A_1 \vee A_2 \vee ... \vee \ C \ \vee ... \vee A_n \qquad B_1 \vee B_2 \vee ... \vee \neg C \vee ... \vee B_m}{A_1 \vee A_2 \vee ... \vee A_n \vee B_1 \vee B_2 \vee ... \vee B_m}$$

Dada una cláusula que contiene la literal C y otra cláusula que contiene la literal $\neg C$, se puede inferir una nueva que consta de las literales de dichas cláusulas sin el par complementario. Al resultado se le denomina **resolvente**.

Es muy importante tener en cuenta que las cláusulas son **conjuntos** de literales, por lo que no puede haber dos ocurrencias de una misma literal en una cláusula. De modo que, el resolvente debe seguir cumpliendo la definición de conjunto (no debe haber literales repetidas).

A las cláusulas que constan de una sola literal se les llama unitarias. Así, si al aplicar resolución binaria una de las dos cláusulas involucradas es unitaria, entonces el resolvente contendrá las literales restantes de la otra cláusula.

El resolvente de dos cláusulas unitarias contrarias es conocido como *cláusula vacía*, pues no contiene ninguna literal. Con esto, obtener una cláusula vacía como resultado indica que el par de cláusulas sobre el que se aplica la regla no es satisfacible.

$$\frac{A \quad \neg A}{\square}$$

Es importante destacar que la regla solo permite que se haga **una resolución a la vez**. En otras palabras, sólo es posible aplicar la regla sobre un par de literales a la vez.

Ejercicios

Para poder implementar la resolución binaria, se necesita primero lo siguiente:

- `type Literal = Prop`
  Crear un sinónimo `Literal`, que será igual a `Prop` por simplicidad. Aunque sólo deberían ser variables o negaciones de variables.
- `type Clausula = [Literal]`
  Crear un sinónimo `Clausula`, que representará las cláusulas como conjunto de literales.

1. `clausulas :: Prop -> [Clausula]`
   Definir la función `clausulas` que dada una fórmula en FNC, devuelve una lista con las cláusulas que la forman. **[2 puntos]**
2. `resolucion :: Clausula -> Clausula -> Clausula`
   Definir la función `resolucion` que dadas dos cláusulas, devuelve el resolvente obtenido después de aplicar la regla de resolución binaria. Se puede asumir que se puede obtener un resolvente a partir de los argumentos. **[2 puntos]**

3.4. Algoritmo de saturación

La resolución binaria es de gran importancia, pues proporciona un método para decidir la satisfacibilidad de una fórmula. No obstante, por sí sola no es un procedimiento algorítmico, pues se sabe que es posible decidir la satisfacibilidad de dos cláusulas usando la resolución binaria; sin embargo, no se indica cómo decidir la satisfacibilidad de una fórmula de la lógica proposicional. Para ello se hace uso de un **algoritmo de saturación**.

Para describir el algoritmo, se necesita tener en cuenta lo siguiente:

- **Función R:** Sea S un conjunto de cláusulas. Definimos $R(S)$ como el conjunto resultado de la unión de S con el conjunto de todos los posibles resolventes entre cada par de cláusulas de S.

$$R(S) = S \cup \{res(c_n, c_m) : c_n, c_m \in S\}$$

- **Función $\mathbf{Res_n}$** : Se define recursivamente como la n-ésima resolución de S

$$Res_0(S) = S$$
$$Res_{n+1}(S) = R(Res_n(S))$$

Con estos conceptos, se puede describir fácilmente el algoritmo diciendo que existe una $n \in \mathbb{N}$ tal que $Res_n(S)$ cumple alguna de las siguientes condiciones:

- $\square \in Res_n(S)$, lo cual nos indica que la fórmula es insatisfacible.
- $Res_{n-1}(S) = Res_n(S)$, lo que significa que ya no se pueden obtener más resoluciones y que la fórmula es satisfacible.
- Se agotan los recursos computacionales, es indeterminada la satisfacibilidad o insatisfacibilidad de la fórmula.

Con esto en mente, hay que seguir la siguiente serie de pasos para el algoritmo de saturación:

i. Convertir A a su forma normal conjuntiva y determinar el conjunto de cláusulas S de A.

ii. Aplicar "fuerza bruta" calculando todos los resolventes posibles con la función Res para llegar a una n tal que $Res_n(S)$ cumple alguna de las condiciones mencionadas anteriormente.

iii. Dependiendo de la condición cumplida, podemos decir si A es o no es satisfacible.

Ejercicios

1. `hayResolvente :: Clausula -> Clausula -> Bool`
   Definir la función `hayResolvente`, que determina si es posible obtener un resolvente a partir de dos cláusulas. **[1 punto]**

2. `saturacion :: Prop -> Bool`
   Definir la función `saturacion`, que dada una fórmula proposicional, determina si esta es satisfacible o no usando el algoritmo de saturación. **[2 puntos]**

¡Buena suerte a todos! ☺☺☺

Nota: Para mayor referencia teórica de los conceptos que se usan en esta práctica, pueden consultar la nota 3 del curso.